

FireWatch: A Drone-Ready Wildfire Risk Prediction and Fire/Smoke Detection System

2026

B.Sc. Thesis

COMPUTER ENGINEERING

MOHANAD EHAB QASEM MOHAMMED AL-HAJJ

AHMED NAJIB AHMED ABDO

ABDULRAHMAN MOHAMMED AHMED ALHALQI

FireWatch: A Drone-Ready Wildfire Risk Prediction and Fire/Smoke Detection System

MOHANAD EHAB QASEM MOHAMMED AL-HAJJ

AHMED NAJIB AHMED ABDO

ABDULRAHMAN MOHAMMED AHMED ALHALQI

Karabük University

Faculty of Engineering

Department of Computer Engineering

Prepared as a B.Sc. Thesis

KARABÜK

May 2026

I approve that the thesis titled

“FireWatch: A Drone-Ready Wildfire Risk Prediction and Fire/Smoke Detection System”

prepared by

MOHANAD EHAB QASEM MOHAMMED AL-HAJJ,
AHMED NAJIB AHMED ABDO, and
ABDULRAHMAN MOHAMMED AHMED ALHALQI

has been approved as a B.Sc. Thesis in the Department of Computer Engineering, Faculty
of Engineering, Karabük University.

Date: ../../2026

Ammar Mohammed Ali AL-QADASI
Thesis Advisor,
Karabük University
Department of Computer Engineering

Jury Member Name
Department / University

Jury Member Name
Department / University

ABSTRACT

FireWatch is a drone-ready wildfire risk prediction and fire/smoke detection prototype developed for the Karabük region in Türkiye. The project addresses the need for an integrated monitoring workflow that combines early weather-based risk awareness with visual evidence from camera sources. The prediction methodology used May–October seasonal Karabük records from 2012 to 2025. Open-Meteo daily weather aggregates were used as accessible model inputs, while the dataset included a processed Fire Weather Index (FWI) target associated with official fire-danger data and used for supervised learning.

The prediction pipeline was designed as a two-stage model. Stage 1 used a HistGradient-BoostingRegressor to estimate continuous FWI from a locked 34-feature schema. Stage 2 used a RandomForestClassifier as a safety-oriented high-risk classifier for days where FWI is greater than or equal to 35. For visual monitoring, a custom YOLOv8 model, best.pt, was used to detect fire and smoke from live camera or drone-ready sources.

The system was implemented using a FastAPI backend, SQLite persistence, scheduled risk checks, and a React/TypeScript dashboard. The dashboard integrates risk decisions, live monitoring feeds, detection alerts, model/system information, workflow explanation, and map context. The results show that FireWatch can integrate prediction, detection, persistence, and dashboard-based decision support into one regional wildfire monitoring prototype. The system remains an academic prototype and does not claim official emergency deployment or full autonomous drone operation.

Key Words: wildfire prediction, fire detection, smoke detection, Fire Weather Index, YOLOv8, drone-ready monitoring, decision-support system

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our thesis advisor, Ammar Mohammed Ali AL-QADASI, for his guidance, support, and constructive feedback throughout the development of this project. His academic direction helped us improve the technical structure and presentation of the FireWatch system.

We also thank the Department of Computer Engineering, the Faculty of Engineering, and Karabük University for providing the academic environment and resources needed to complete this B.Sc. thesis. We are grateful to our families for their patience, encouragement, and support during our studies.

Finally, we thank each member of the project team for their collaboration and shared effort. We also acknowledge the open-source tools, software libraries, and public data resources that supported the implementation, testing, and documentation of this project.

DECLARATION

“I declare that all information in this thesis has been obtained and presented in accordance with academic rules and ethical principles. I also declare that all sources, ideas, data, figures, tables, and results that are not original to this work have been properly cited and referenced in accordance with the required academic rules and ethical principles.”

MOHANAD EHAB QASEM MOHAMMED AL-HAJJ
AHMED NAJIB AHMED ABDO
ABDULRAHMAN MOHAMMED AHMED ALHALQI

Contents

Abstract	ii
Acknowledgement	iii
Declaration	iv
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	2
1.3 Importance of the Study	2
1.4 Research Question	3
1.5 Aim and Objectives	3
1.6 Scope of the Project	3
1.7 Main Contributions	4
1.8 Thesis Organization	4
2 Literature Review	5
2.1 Fire Prediction Studies	5
2.2 Fire Detection Studies	7
2.3 Limitations of Reviewed Studies	8
2.4 Comparative Summary	8
3 Methodology	11
3.1 Methodology Overview	11
3.2 Study Region and Data Sources	12
3.3 Data Collection Period and Seasonal Filtering	13
3.4 Daily Weather Aggregation and Model Input Alignment	13
3.5 Data Cleaning and Alignment	14
3.6 Feature Engineering	14
3.7 Stage 1: Wildfire Risk Regression Model	16
3.8 Stage 2: High-Risk Safety Classifier	16
3.9 Fire and Smoke Detection Methodology	17
3.10 Operational Decision Workflow	18

3.11	Methodology Summary	19
4	System Design and Implementation	20
4.1	System Architecture Overview	20
4.2	Backend and API Design	21
4.3	Database and Persistence Design	22
4.4	Prediction and Detection Integration	24
4.5	Frontend Dashboard Design	25
4.6	Scheduler and Operational Run Design	26
4.7	Drone-Ready Implementation	27
4.8	Implementation Summary	28
5	Results and Discussion	29
5.1	Stage 1 Regression Results	29
5.2	Stage 2 High-Risk Classification Results	31
5.3	YOLOv8 Fire and Smoke Detection Results	33
5.4	Dashboard and System Validation Results	37
5.5	Limitations	37
5.6	Discussion	39
6	Conclusion and Future Work	40
6.1	Conclusion	40
6.2	Future Work	41
	References	43

List of Figures

3.1	End-to-end FireWatch workflow from weather input and visual monitoring to prediction, detection alerts, persistence, and dashboard review.	12
3.2	Grouped view of the 34 FireWatch prediction features used by the deployed Stage 1 model.	15
3.3	Prediction-stage workflow showing feature construction, Stage 1 FWI regression, Stage 2 high-risk classification, and final decision logic.	17
3.4	Detection-stage workflow for camera or drone-ready frames, YOLOv8 fire/smoke inference, alert creation, and dashboard review.	18

4.1	FireWatch system architecture connecting the React/TypeScript dashboard, FastAPI backend, prediction services, YOLOv8 monitoring services, scheduler, drone-ready service, and SQLite persistence.	21
4.2	SQLite persistence design showing the main runtime tables used by FireWatch: prediction history, system state, and detection alerts.	24
4.3	Detection alert workflow from a YOLOv8 detection event to persistent unread alerts, operator review, read/unread state, and soft deletion.	25
4.4	Runtime sequence for scheduled risk checks, feature generation, two-stage inference, SQLite storage, and dashboard update.	27
5.1	Stage 1 actual versus predicted FWI comparison for candidate regression models.	30
5.2	Stage 1 residual analysis showing prediction error behaviour across FWI values.	30
5.3	Regression benchmark comparison using R^2 , RMSE, and MAE.	30
5.4	Stage 2 precision–recall comparison for imbalanced high-risk classification.	31
5.5	Final Stage 2 high-risk classifier confusion matrix using the report threshold of 0.10.	32
5.6	Final Stage 2 classifier metrics at threshold 0.10.	32
5.7	Feature importance evidence for the enriched Random Forest classifier, highlighting the contribution of the support features.	33
5.8	YOLOv8 training and validation curves for loss and detection metrics.	34
5.9	Precision–recall curve for fire and smoke detection.	35
5.10	F1-confidence curve used to inspect detection threshold behaviour.	35
5.11	Normalized confusion matrix for fire, smoke, and background predictions.	36
5.12	Example validation images comparing ground-truth labels with YOLOv8 predictions.	36

List of Tables

2.1	Reported limitations of the wildfire prediction and detection studies reviewed in this chapter.	9
2.2	Very short comparative summary of selected wildfire prediction and detection studies.	10
5.1	Stage 1 regression performance for continuous FWI prediction.	29
5.2	Final Stage 2 high-risk classification result at probability threshold 0.10. .	31
5.3	YOLOv8 fire/smoke detection metrics from the uploaded training and evaluation output.	34
5.4	System validation evidence from the repository and local verification. . .	38

Chapter 1

Introduction

1.1 Background

Forest fires are one of the most serious natural hazards affecting Türkiye. Every year, thousands of hectares of forest are damaged, threatening human lives, wildlife, critical infrastructure, and the national economy [1]. The combination of hotter summers, prolonged drought periods, and human activities has increased both the frequency and severity of wildfires [2, 3].

Traditional wildfire monitoring in Türkiye still relies heavily on fixed lookout towers, ground patrols, and reports from local communities. Although these methods are useful, they suffer from several limitations: they are labor-intensive, dependent on human vigilance, and often detect fires only after they have grown to a dangerous size [4]. At the same time, recent advances in unmanned aerial vehicles (UAVs), low-cost sensors, and artificial intelligence (AI) provide a new opportunity to move from reactive firefighting to proactive, risk-driven monitoring [5].

This project focuses on FireWatch: A Drone-Ready Wildfire Risk Prediction and Fire/Smoke Detection System for the Karabük region. The final system combines daily wildfire risk prediction from 2012–2025 seasonal Karabük records, live fire/smoke detection using a custom YOLOv8 detector, persistent alert storage, and dashboard-based monitoring. The system also includes a drone-ready layer for mock, Tello, and demo support, but it does not claim to be a fully autonomous production drone deployment.

1.2 Problem Statement

Despite the increasing impact of wildfires in Türkiye, current monitoring and early-warning practices still show important gaps:

- Fire risk is not always quantified in a continuous and operational form that can support daily monitoring decisions for the target region [1].
- Drone and camera systems, when used, are usually operated manually rather than being prioritized or recommended based on risk levels [6].
- Many existing camera systems only provide live video without intelligent fire/smoke detection, which means they still depend on human operators [5].
- Historical data on risk, detections, and responses are not fully integrated into a single platform that can support post-event analysis and future planning.

As a result, fires may be detected late, high-risk periods may not receive enough attention, and decision-makers may lack an integrated view of risk, detection, and response. In the final FireWatch system, this gap is addressed by combining risk prediction, live fire/smoke detection, persistent alerts, and a dashboard view.

Prediction alone cannot confirm whether fire or smoke is already present, while visual detection alone does not provide the daily risk context needed for preparedness and prioritization. Operators therefore need an integrated system that connects risk estimates, visual evidence, alert history, and system status in one workflow.

1.3 Importance of the Study

Developing an intelligent wildfire monitoring system has clear real-world value. Earlier and more reliable detection of fires can reduce burned area, economic losses, and ecological damage [5]. A risk-driven monitoring approach can help forestry authorities and operators focus limited resources on the most vulnerable periods and locations instead of monitoring large areas uniformly.

The drone-ready layer adds another engineering dimension by preparing the system for future patrol support. In the current implementation, this layer is limited to mock, Tello, and demo support and should be understood as a prototype direction rather than a field-tested autonomous drone deployment.

1.4 Research Question

The main research question of this thesis is:

How can a drone-ready wildfire monitoring system combine daily wildfire risk prediction, live fire/smoke detection, persistent alert management, and dashboard-based decision support for the Karabük region?

1.5 Aim and Objectives

The aim of this project is to design and implement FireWatch as a drone-ready wildfire risk prediction and fire/smoke detection prototype for the Karabük region.

To achieve this aim, the project sets the following specific objectives:

1. Predict FWI and high risk days using machine learning instead of traditional calculation methods.
2. Flag high-risk days early using a clear threshold based on historical fire records.
3. Detect and confirm fire/smoke in real time using a custom YOLOv8 model.
4. Integrate prediction, detection, alerts, and dashboard monitoring into one unified system.

Each objective directly addresses limitations identified in the problem statement and can be evaluated using measurable indicators such as prediction performance, detection performance, response time, persistence behavior, testing results, and dashboard usability.

1.6 Scope of the Project

This project is scoped as a pilot implementation for **Karabük Province** (Türkiye) and its surrounding forested areas. Karabük is located in the Western Black Sea region and is characterized by extensive forest coverage and mixed terrain, where seasonal heat, dry winds, and human activity near roads and settlements can elevate wildfire risk.

The project focuses on the period from 2012 to 2025, which includes 14 fire seasons and provides a rich dataset for training and testing the prediction model.

1.7 Main Contributions

The main contributions of this thesis are summarized as follows:

- An end-to-end FireWatch prototype that combines wildfire risk prediction, fire/smoke detection, alert storage, and dashboard monitoring.
- A 34-feature prediction pipeline for regional daily wildfire risk estimation.
- A two-stage prediction model using a Stage 1 HistGradientBoostingRegressor and a Stage 2 RandomForestClassifier safety classifier.
- A custom YOLOv8 fire/smoke detector using best.pt as the active detection model.
- A dashboard that includes Live Monitoring, Risk Decision, Risk Map, Detection Alerts, System and Model, and System Flow views.

1.8 Thesis Organization

Chapter 2 reviews related studies on wildfire prediction and fire/smoke detection, including the limitations of reviewed studies and a comparative summary. Chapter 3 explains the methodology, including data preparation, feature engineering, prediction modeling, and visual detection methodology. Chapter 4 presents the system design and architecture, including the backend, frontend, database, scheduler, and drone-ready layer.

Chapter 5 presents the results and discussion, including prediction results, YOLOv8 detection evidence, dashboard behaviour, validation, limitations, and interpretation. Chapter 6 concludes the thesis and describes future work.

Chapter 2

Literature Review

This chapter reviews recent work on wildfire prediction and detection that is most relevant to the FireWatch pipeline, where weather-based risk triggers drone missions and a vision model confirms smoke or fire from aerial imagery.

2.1 Fire Prediction Studies

Kondylatos et al. design deep learning models based on LSTM and ConvLSTM networks to forecast short-term wildfire danger from spatio-temporal environmental variables, including weather and moisture-related inputs [7]. Their ConvLSTM configuration shows clear improvements over traditional index-based baselines, with reported AUROC values around 0.93 and F1-scores near 0.8. They also apply model interpretability tools to highlight influential predictors. The main limitation is that the models must be retrained and validated for each new region. For FireWatch, this kind of ConvLSTM risk model can strengthen the trigger that decides when drones should be dispatched.

Shmuel and Heifetz introduce a machine-learning fire weather index (MLFWI) trained on global ignition-related data using meteorological, vegetation and contextual features [8]. The index is built with gradient-boosted trees (XGBoost) and achieves very high performance compared to traditional indices, with ROC-AUC values close to 0.99 and accuracy near 0.96. Its strength is the ability to capture non-linear interactions between drivers such as heat, drought and wind; its weakness is the need for large, high-quality datasets and careful regional calibration. In FireWatch, an MLFWI-style score can be converted directly into zone-level risk values that drive the risk map and drone scheduling.

Zaidi studies wildfire occurrence in Algerian forests with several machine learning classifiers based on weather and FWI-related inputs, using dimensionality reduction to handle correlated variables [9]. An artificial neural network (ANN) model reports accuracy around 0.967 and F1-score about 0.971 on the considered dataset. This shows that high predictive performance is possible even with limited local data, although generalization outside the specific region and period requires additional data. For FireWatch, this supports the idea that zone-level ignition prediction is feasible for local forests, helping define thresholds that automatically trigger drone missions.

Li et al. combine IoT-style local sensing with neural networks to estimate fire danger indices, focusing on peatland-relevant variables such as humidity, temperature and other microclimate factors [10]. They report strong agreement with reference indices, with mean squared error of about 1.116 and correlation near 0.89. A key advantage is the ability to capture fine-scale conditions that may not appear in coarse-grained weather products; however, the approach predicts an index rather than ignition directly and requires on-site deployment and maintenance of sensors. In FireWatch, such ground sensors could complement weather APIs to refine risk scores before drone dispatch.

Sengupta and Woodford propose an explainable machine learning workflow for wildfire occurrence and burned-area prediction [11]. They evaluate multiple models and use SHAP-based analysis to interpret feature importance, reporting very strong classification performance (F1 up to roughly 0.99) and good burned-area regression metrics (e.g. RMSE around 1.9, MAE about 1.1) on benchmark datasets. The benefit is a systematic model-selection and interpretation framework; the drawback is reliance on relatively small benchmark datasets that may not fully reflect operational variability. For FireWatch, this work helps justify feature choices and provides a way to explain “why risk is high” to operators alongside automated drone missions.

Anastasiou et al. move beyond ignition and focus on forecasting wildfire spread and perimeter with deep learning segmentation models that use multi-source, multi-day context [12]. They report measurable gains (around +5% in F1 and IoU) when including a wider temporal window in the input. This shows that spread-aware forecasting is possible, but it also raises the data and processing requirements and again requires regional validation. In FireWatch, such spread forecasting becomes important after a fire is detected: once drones confirm a fire, spread predictions could support response planning and prioritization.

2.2 Fire Detection Studies

Bouguettaya et al. provide a survey of UAV-based wildfire detection methods that use deep-learning computer vision techniques [13]. They categorize approaches by sensor type (RGB, infrared, multispectral) and by model family (image classification, YOLO-style object detection, semantic segmentation), and they discuss challenges such as dataset scarcity, domain shift and edge deployment constraints. Although the paper does not introduce new experiments, it offers a practical roadmap for selecting drone-vision methods. This directly informs FireWatch’s choice of a lightweight, real-time detector for onboard or edge processing on drones.

Mukhiddinov et al. optimize YOLOv5 architectures for early smoke detection in UAV imagery, including anchor-box tuning and multi-scale enhancements [14]. On their dataset they report mAP@0.5 around 73.6%, with improved sensitivity to small smoke plumes. The method shows that careful architecture tuning can significantly improve early smoke detection, although the absolute performance and robustness still depend on dataset diversity. For FireWatch, this work is relevant because confirming small smoke plumes from drones is critical when the weather-based module flags a high-risk day.

Ramadan et al. combine IoT sensing with AI-powered drones in an end-to-end early detection framework [15]. Ground sensors monitor environmental conditions; when thresholds are exceeded, a UAV is launched and an onboard CNN verifies the presence of fire. They report very high sensor accuracy (greater than 99%) and fire detection performance for the CNN, with rapid alerting. The strength of this approach is multi-modal confirmation, while the drawback is deployment complexity in terms of infrastructure, maintenance and flight constraints. This study closely matches FireWatch’s concept of prediction/trigger → drone dispatch → vision confirmation.

Abramov et al. propose a real-time UAV video pipeline that combines stabilization and deblurring, region masking and deep detection/segmentation to improve fire recognition in cluttered scenes [16]. They report detector performance around mAP $\approx$ 78% and segmentation F1-scores near 97% in their setup, with gains when confusing regions such as the sky are masked out. The approach is robust to drone motion and reduces false alarms, but it increases pipeline complexity and compute requirements. For FireWatch, this suggests a design where light pre-processing (e.g. masking regions unlikely to contain fire) is combined with fast detectors for live drone feeds.

Mamadmurodov et al. develop a hybrid YOLO-based model with attention mechanisms and domain-aware preprocessing aimed at early forest fire detection [17]. They report mAP@0.5 of about 93.1% with high precision and recall at real-time speeds on GPU. The

main advantage is the combination of high accuracy and efficiency; the key limitation is that detection is still frame-based, without explicit temporal modeling. For FireWatch, such a model is attractive as a practical starting point for reliable smoke or fire detection on the edge.

He et al. evaluate several deep object detection architectures for smoke recognition in forest fire scenarios [18]. After extensive training on diverse wildfire imagery, they obtain strong performance, with mAP@0.5 around 90.1% and high average precision for smoke classes. The benefits are high smoke sensitivity and robust detection; the downsides include heavier training and computation requirements and uncertain feasibility on very constrained onboard hardware. For FireWatch, this work supports the requirement of high-confidence smoke detection to validate high-risk conditions predicted by the weather module.

2.3 Limitations of Reviewed Studies

To clarify the motivation behind the FireWatch design choices, Table 2.1 summarizes the main disadvantages reported across the reviewed wildfire *prediction* and *detection* studies. These limitations highlight recurring challenges such as regional transferability, dependence on dataset quality and diversity, and the practical constraints of deploying AI models on UAV or edge hardware.

2.4 Comparative Summary

Table 2.2 provides a compact comparison of the main prediction and detection studies reviewed in this chapter. It highlights the variety of methods (from ConvLSTM risk forecasters to YOLO-based smoke detectors), the performance levels reported and the parts of the FireWatch pipeline that each study informs.

Table 2.1: Reported limitations of the wildfire prediction and detection studies reviewed in this chapter.

Ref.	Study	Key limitations (disadvantages)
[7]	Kondylatos et al. (2022)	• Region-specific (Eastern Mediterranean); transferability is limited. • Focuses on next-day danger only (not multi-day spread/burned area).
[8]	Shmuel & Heifetz (2023)	• Requires large datasets and substantial computational resources. • Limited multi-year/region validation; real-world robustness needs more evidence.
[9]	Zaidi (2023)	• Small dataset (one year, few regions); generalisation is uncertain. • Uses mainly meteorology/FWI; omits fuel, topography, and human factors.
[10]	Li et al. (2023)	• Single-site (peatland) and short monitoring period; limited generalisation. • Predicts FWI rather than observed fire events (operational linkage is indirect).
[11]	Sengupta & Woodford (2025)	• Built on small regional benchmarks with limited variables. • Scalability to operational, large-area deployments remains unclear.
[12]	Anastasiou et al. (2025)	• Calibrated for Mediterranean conditions; may need re-tuning elsewhere. • Multi-day/multi-channel inputs increase computational cost.
[13]	Bouguettaya et al. (2022)	• Survey-only (no new dataset/model for direct benchmarking). • Coverage mainly up to 2021; misses newer YOLO/transformer and multimodal systems.
[14]	Mukhiddinov et al. (2022)	• Notes dataset quality/size issues that limit real-world generalisation. • Reported performance is modest relative to stronger baselines.
[15]	Ramadan et al. (2024)	• Limited large-scale field validation under diverse conditions. • Deployment constraints (LoRa reliability/RF interference + UAV regulations).
[16]	Abramov et al. (2024)	• Sensitive to fog/snow and camera orientation (horizon-line dependency). • Complex pipeline increases compute needs for low-power UAV deployment.
[17]	Mamadmurodov et al. (2025)	• Primarily static-image evaluation; does not model temporal evolution. • Limited evidence on long UAV video streams and real-time constraints.
[18]	He et al. (2025)	• Struggles in low-contrast/cluttered scenes and overlapping smoke-flame regions. • Limited evaluation on edge hardware and real UAV video deployments.

Table 2.2: Very short comparative summary of selected wildfire prediction and detection studies.

Study	Year	Method	Main contribution	Performance
Kondylatos et al. [7]	2022	ConvLSTM	Daily wildfire danger forecasting from spatio-temporal inputs	AUROC $\approx$ 0.93; F1 $\approx$ 0.8
Shmuel & Heifetz [8]	2023	XGBoost (MLFWI)	Machine-learning-based fire weather index	AUC $\approx$ 0.99; Acc $\approx$ 0.96
Zaidi [9]	2023	ANN	Local wildfire ignition prediction for Algerian forests	Acc $\approx$ 0.967; F1 $\approx$ 0.971
Li et al. [10]	2023	MLP (IoT)	Index prediction from on-site sensor measurements	MSE = 1.116; $R \approx$ 0.89
Sengupta & Woodford [11]	2025	XAI + HPO	Explainable models for occurrence and burned-area	F1 up to $\sim$ 0.99; RMSE $\approx$ 1.9
Anastasiou et al. [12]	2025	3D U-Net / ViT	Wildfire spread and perimeter forecasting	About +5% F1/IoU gain
Bouguettaya et al. [13]	2022	Review	UAV + DL wildfire detection survey	Qualitative synthesis
Mukhiddinov et al. [14]	2022	Optimized YOLOv5	Early smoke detection in UAV images	mAP@0.5 $\approx$ 73.6%
Ramadan et al. [15]	2024	IoT + UAV CNN	End-to-end early forest fire detection	Accuracy in $\sim$ 99% range
Abramov et al. [16]	2024	Video + YOLO / Seg.	Robust UAV video fire detection pipeline	mAP $\sim$ 78%; seg. F1 $\sim$ 97%
Mamadmurodov et al. [17]	2025	YOLO + Attention	Efficient early forest fire detection	mAP@0.5 $\approx$ 93.1%
He et al. [18]	2025	Object detection	Forest fire smoke recognition	mAP@0.5 $\approx$ 90.1%

Chapter 3

Methodology

3.1 Methodology Overview

The methodology of FireWatch is built around two complementary forms of wildfire monitoring. The first is weather-based risk prediction, which estimates daily fire-weather danger for the Karabük region. The second is visual fire/smoke detection, which provides camera-based evidence when fire or smoke appears in a monitoring feed. These two modules are methodologically separate: the prediction model estimates fire-weather risk, while the detection model identifies visible fire or smoke evidence.

The prediction workflow begins with weather variables collected for Karabük from Open-Meteo, while the processed dataset includes a Fire Weather Index (FWI) target associated with official CEMS/Copernicus fire-danger data. The prepared dataset stores this supervised learning target in the column FWI. The repository stores the processed FWI target used for modelling, while the raw external download workflow is treated as a data-preparation step rather than a runtime dependency. The project does not calculate the full Canadian FWI equation chain at runtime. Instead, the FWI value is used as the training target, and the deployed model learns to approximate it from accessible weather variables and engineered temporal features. This choice keeps the operational pipeline practical while anchoring the model to an established fire-danger indicator.

The complete workflow is as follows: weather records are collected and aligned by date, the May–October seasonal window is selected, raw weather variables are transformed into a locked 34-feature schema, a Stage 1 regression model predicts continuous FWI, and a Stage 2 safety classifier estimates whether the day should be treated as high risk. The resulting decision is stored and shown on the dashboard. In parallel, the YOLOv8

fire/smoke detector processes camera or drone-ready video sources and creates persistent detection alerts when fire or smoke is detected.

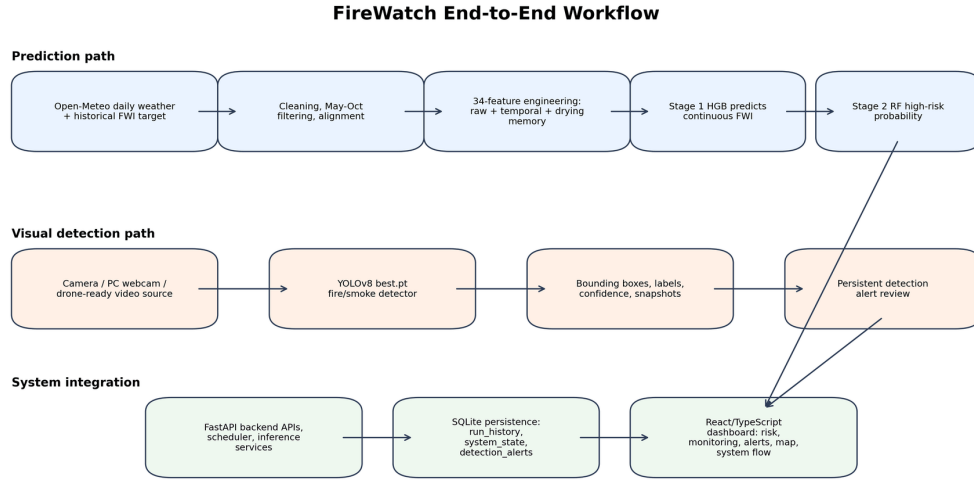


Figure 3.1: End-to-end FireWatch workflow from weather input and visual monitoring to prediction, detection alerts, persistence, and dashboard review.

Figure 3.1 summarizes the full methodology. It shows that weather-based prediction and YOLOv8 visual detection are connected inside the same system, but they answer different operational questions.

3.2 Study Region and Data Sources

The study region is Karabük, Türkiye, with the system configuration using the coordinates 41.2061° latitude and 32.6204° longitude for model-input weather collection. Karabük was selected as a regional pilot because the project aims to support local wildfire preparedness rather than produce a generic non-local risk model.

Weather variables are obtained from Open-Meteo. The data pipeline separates historical model-input preparation from operational forecast-based inference: historical archive data are used for past records, while forecast data can be used when the target date falls on or after the current Istanbul-local date. The weather variables used by the model are daily aggregate values rather than display-only live weather values. This separation is important because the dashboard can show current weather for operator awareness, but the prediction model must receive inputs that match the training dataset semantics.

The supervised learning target is the FWI value stored in the dataset as FWI. The final FireWatch methodology treats this target as the daily fire-danger signal used during training.

The Canadian FWI system is an established fire-danger rating framework [19]; however, FireWatch does not reproduce the full chain of FWI sub-index equations during runtime. Instead, the target teaches the model how daily weather and recent drying patterns relate to FWI.

3.3 Data Collection Period and Seasonal Filtering

The processed FireWatch dataset covers seasonal records from 1 May 2012 to 31 October 2025. The processed file contains 2,576 daily rows and uses the months May, June, July, August, September, and October. This confirms that the training and evaluation data are restricted to the warm-season period most relevant to wildfire risk in the project scope.

Seasonal filtering is used for methodological reasons. Including winter and low-risk months would add many examples whose weather patterns are less informative for fire-season decisions. This could make the model appear more stable while weakening its focus on the period when wildfire preparedness is operationally important. Restricting the dataset to May–October keeps the learning problem aligned with the dashboard’s purpose: supporting daily fire-season monitoring for Karabük.

3.4 Daily Weather Aggregation and Model Input Alignment

The current model-input pipeline does not use noon-only extraction. The Open-Meteo input pipeline requests daily aggregate variables with semantics that match the training CSV. These daily inputs are: `temperature_2m_max`, `relative_humidity_2m_min`, `wind_speed_10m_max`, `precipitation_sum`, `cloud_cover_mean`, `shortwave_radiation_sum`, and `et0_fao_evapotranspiration`. These variables are mapped into the model schema as `temperature`, `rh`, `ws`, and `precip` where needed. The `soil_moisture_0_to_7cm_mean` variable is added separately through the soil-moisture resolver.

This design converts weather data into one representative daily model row. It does not attempt to reconstruct every hourly fire-weather observation or the full Canadian FWI state chain. The methodological reason is consistency: Stage 1 was trained on daily model-input variables, so scheduled and manual risk checks must use the same daily aggregate meanings. The official FWI target is daily and is used only as a supervised training label; it is not included as a runtime input feature.

3.5 Data Cleaning and Alignment

The prepared dataset is organised as daily records and aligned using the date column as the main temporal key. Additional date-related columns, such as day, month, and year, are retained to support seasonal filtering and calendar-based feature engineering. Before model training and inference, the records are sorted chronologically so that rolling and temporal-memory features are computed in the correct order.

Column names are standardised to match the model feature schema. For example, weather variables such as temperature, relative humidity, wind speed, and precipitation are represented consistently as `temperature`, `rh`, `ws`, and `precip`. This standardisation is important because the same feature schema must be used during both model training and operational inference. The high-risk classification target is derived from the official FWI value as `target_ge_35`, where a day is considered high risk when $\text{FWI} \geq 35$.

Target leakage is controlled during temporal feature construction. Rolling and exponentially weighted features are computed from shifted weather series so that they describe previous conditions rather than accidentally including the same day's target-related information. As a result, rolling rainfall, humidity, temperature, wind, EWMA, and dry-day features represent historical weather memory. The official FWI value is used for supervised training and evaluation, but it is not included as one of the 34 runtime input features.

3.6 Feature Engineering

FireWatch uses a locked schema of 34 input features for the prediction model. These features are designed to approximate the environmental behaviour represented by official FWI while relying on accessible weather variables and locally computed temporal features. The schema contains eight raw daily weather features and twenty-six engineered features.

The raw daily weather inputs are: `temperature`, `rh`, `ws`, `precip`, `cloud_cover_mean`, `shortwave_radiation_sum`, `et0_fao_evapotranspiration`, and `soil_moisture_0_to_7cm_mean`. These variables represent immediate fire-weather conditions such as heat, humidity stress, wind exposure, rainfall, solar energy, evaporation demand, and near-surface soil moisture.

Seasonal and cyclical features are represented by `day_sin`, `day_cos`, `month_sin`, and `month_cos`. These features allow the model to learn seasonal patterns without treating calendar day or month as a simple linear value.

Derived fire-weather features include `vpd`, `fuel_drying_rate`, `hdw`, `wind_squared`, and `dew_point`. These variables describe humidity stress, drying potential, wind-driven spread potential, and the relationship between temperature and atmospheric moisture. They help the model represent fire-relevant physical behaviour without calculating the full FWI equation chain during runtime.

Rolling history features describe recent weather memory. They include `precip_sum_3d`, `precip_sum_7d`, `precip_sum_30d`, `t_mean_3d`, `t_mean_7d`, `rh_min_3d`, `rh_min_7d`, `rh_mean_3d`, `rh_mean_7d`, `ws_mean_3d`, `ws_mean_7d`, `ws_max_3d`, and `ws_max_7d`. These variables capture accumulated rainfall memory, persistent heat, low-humidity periods, and recent wind exposure.

Exponentially weighted memory features are `ewma_t`, `ewma_rh`, and `ewma_precip`. They assign stronger weight to recent weather while still preserving short-term historical influence. The final dryness persistence feature is `consecutive_dry_days`, which measures how long the recent period has remained dry. Together, these feature groups allow the model to represent short-term conditions, accumulated drying, seasonality, and recovery after rainfall.

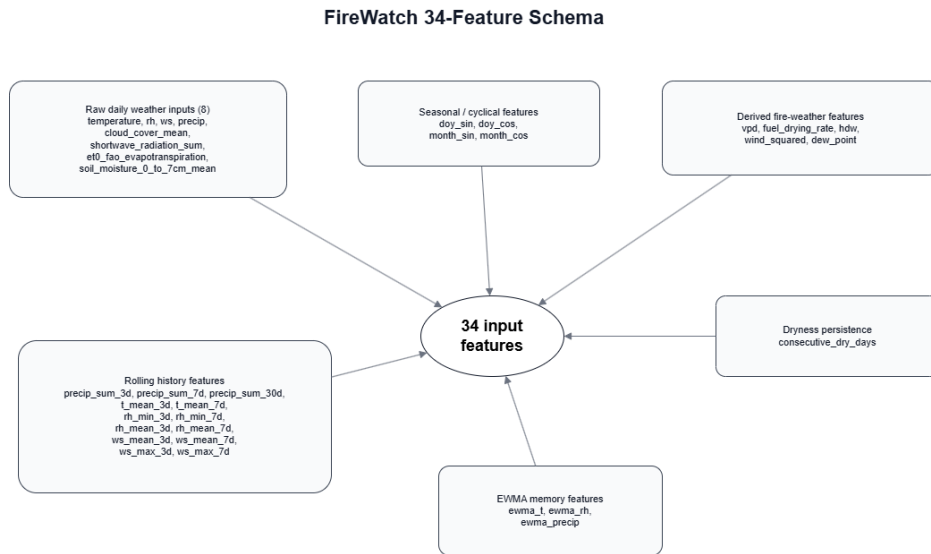


Figure 3.2: Grouped view of the 34 FireWatch prediction features used by the deployed Stage 1 model.

The feature groups in Figure 3.2 keep the prediction schema explainable. They also help ensure that training and runtime inference use the same model inputs.

3.7 Stage 1: Wildfire Risk Regression Model

Stage 1 is the core numerical prediction model in the FireWatch pipeline. It uses a `HistGradientBoostingRegressor` to predict the continuous FWI target from the 34 engineered input features. The output of this model is `predicted_fwi`, which represents the estimated daily fire-weather danger for the target date.

Regression is suitable for Stage 1 because FWI is a continuous intensity index rather than a binary class label. Predicting a continuous value preserves information about the severity of fire-weather conditions and shows how close a day is to the high-risk boundary. This value can also be displayed on the dashboard as part of the risk explanation for operators.

The model is implemented within a scikit-learn pipeline that handles missing values consistently using median imputation. During training, sample weighting is used so that mid-FWI and high-FWI cases receive greater attention than low-risk cases. This is important because high-risk conditions are operationally more critical, even if they occur less frequently than low-risk days.

The stored Stage 1 metadata reports the model configuration, feature count, training/test split, and validation metrics. These metrics are discussed later in the Results and Discussion chapter, while this section focuses on the modelling procedure.

3.8 Stage 2: High-Risk Safety Classifier

Stage 2 is a safety-oriented classification layer built on top of the Stage 1 regression output. It uses a `RandomForestClassifier` trained on Stage 1 out-of-fold predictions together with three support features: `rh`, `ws`, and `fuel_drying_rate`. The full Stage 2 input vector is therefore `predicted_fwi`, `rh`, `ws`, and `fuel_drying_rate`. Its target is `target_ge_35`, where a day is labelled as high risk when $\text{FWI} \geq 35$.

Stage 2 does not replace the Stage 1 regression model. Instead, it supports the final operational decision by focusing on high-risk cases, especially near the decision boundary. The regression output remains the main signal: if `predicted_fwi` is at least 35, the day is classified as high risk. If `predicted_fwi` falls in the grey zone from 28 to below 35, the Stage 2 probability can promote the day to high risk when it reaches the configured probability threshold. The stored threshold is 0.10.

This two-stage structure is used because high FWI conditions are more important from an operational safety perspective than average low-risk predictions. The regression model

preserves the continuous risk intensity, while the classifier adds a decision-support layer focused on avoiding missed high-risk cases. The stored Stage 2 metadata reports the training/test split, probability threshold, and classification metrics. These results are discussed later in the Results and Discussion chapter.

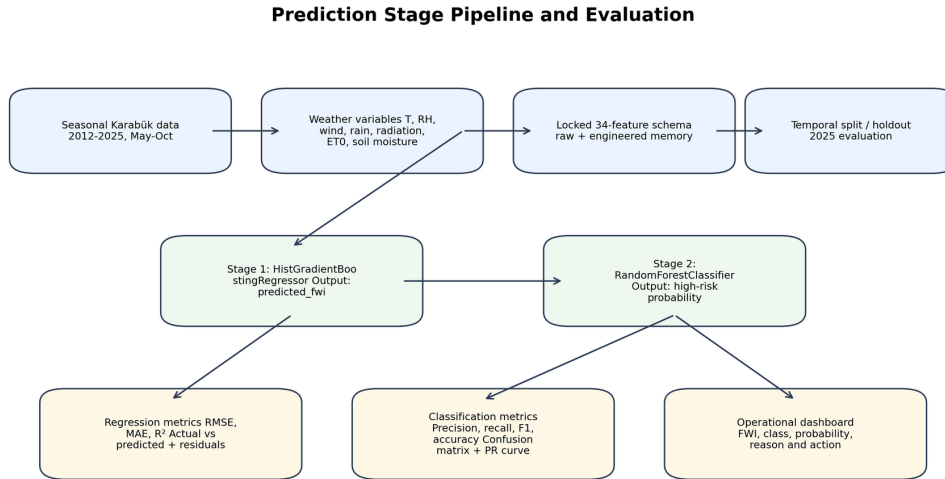


Figure 3.3: Prediction-stage workflow showing feature construction, Stage 1 FWI regression, Stage 2 high-risk classification, and final decision logic.

Figure 3.3 highlights why the prediction stage is not a single binary classifier. Continuous FWI estimation is preserved first, then the high-risk safety layer supports the final operational flag.

3.9 Fire and Smoke Detection Methodology

The visual detection module is separate from the weather-based prediction pipeline. It does not predict FWI and does not modify the wildfire risk model. Its role is to inspect camera frames and identify visible fire or smoke evidence that can be stored and reviewed by an operator.

The active detector is a custom YOLOv8 object detector loaded from `best.pt` as the final fire/smoke detection model. The detector supports two classes: fire and smoke. Runtime labels are normalised so that Fire, fire, flame, and flames map to `fire`, while Smoke and smoke map to `smoke`. Unknown labels are handled safely rather than being forced into the fire class.

YOLOv8 is suitable for this module because it performs object localisation, not only image-level classification. It can produce bounding boxes, confidence values, and class labels on

live camera frames, which are useful for visual confirmation and alert review. In the current prototype, the default detection confidence is configured through `MONITORING_YOLO_CONF` with a default value of 0.4, and the default image size is 640.

The current detection methodology remains frame-based. False positives from bright light, glare, reflections, or visually similar objects remain a known limitation. More advanced robustness methods, such as multi-frame confirmation, glare filtering, bounding-box area filtering, and extended field validation, are left for future work.

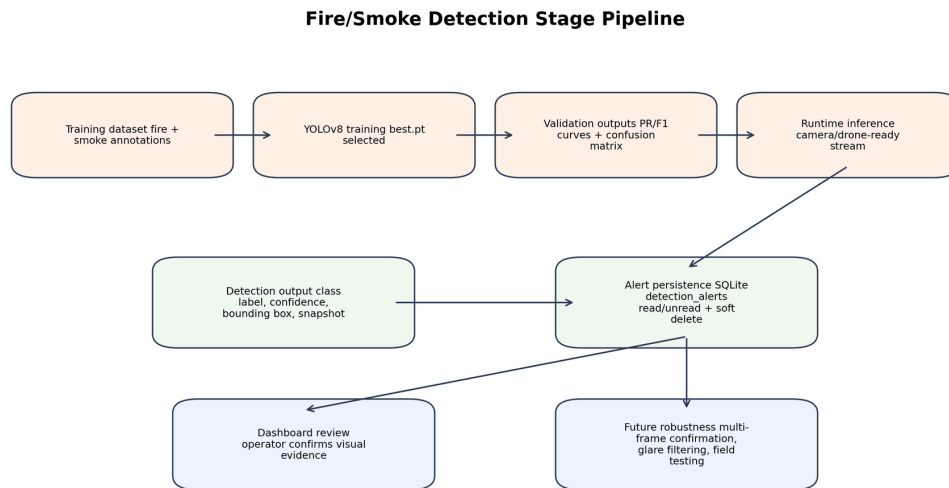


Figure 3.4: Detection-stage workflow for camera or drone-ready frames, YOLOv8 fire/smoke inference, alert creation, and dashboard review.

Figure 3.4 shows the monitoring path as a separate evidence pipeline. It does not write prediction runs or alter FWI estimates.

3.10 Operational Decision Workflow

At runtime, the operational prediction workflow starts by selecting the target date in the Istanbul operational calendar. The system obtains daily Open-Meteo model inputs, builds the 40-day history window needed for rolling and exponentially weighted features, generates a single 34-feature row, and runs the stacked prediction pipeline. The output includes `predicted_fwi`, `high_risk_probability`, a binary high-risk flag, and the decision reason.

Scheduled checks are configured for 09:00, 11:00, and 15:00 Istanbul time. Each scheduled check follows the same data-to-decision path as a manual risk check and stores the result for

dashboard review. The system distinguishes scheduled and manual runs so that operators can audit when a decision was created and why it was created.

The dashboard consumes the prediction output as a decision-support signal. It shows risk information, recent runs, model/system status, and map context. SQLite stores prediction history and detection alerts so that results do not disappear after a page refresh or backend restart. Detection alerts preserve the fire/smoke label and can be reviewed through the Detection Alerts interface.

Drone-ready support is included as an operational layer rather than as a claim of full autonomous deployment. The project supports mock and Tello-oriented configuration, demo patrol behaviour, and camera-stream integration. Physical drone actions remain protected by operator-controlled safety gates, while full autonomous waypoint patrol, regulatory flight approval, and production safety certification remain future work.

3.11 Methodology Summary

The final FireWatch methodology uses a regional, two-stage, decision-support workflow for Karabük. The processed FWI target provides the supervised training signal, while Open-Meteo weather variables and engineered memory features provide practical runtime inputs. Stage 1 estimates continuous FWI, Stage 2 adds a safety layer for high-risk decisions, and the YOLOv8 detector supplies visual fire/smoke evidence through a separate monitoring path.

Chapter 4

System Design and Implementation

4.1 System Architecture Overview

FireWatch is designed as an integrated decision-support prototype for wildfire monitoring in the Karabük regional context. The system combines weather-based risk prediction, visual fire/smoke detection, persistent alert storage, and a dashboard interface into one workflow. Its purpose is not to replace official emergency systems, but to demonstrate how software, machine learning, computer vision, and operator-facing tools can support earlier awareness and more structured monitoring decisions.

The architecture is organised into several cooperating layers. The data and model-input layer prepares daily weather inputs and engineered features. The prediction layer applies the two-stage wildfire risk model, consisting of a Stage 1 `HistGradientBoostingRegressor` and a Stage 2 `RandomForestClassifier`. The visual detection layer uses the active `YOLOv8 best.pt` detector to identify fire and smoke evidence from camera sources. These layers are coordinated through a FastAPI backend, stored through SQLite persistence, and presented through a React/TypeScript dashboard. A drone-ready support layer provides mock, Tello-oriented, and demo workflows for supervised prototype operation.

Prediction and detection are separated by design. The prediction module estimates daily fire-weather risk from weather-derived inputs and produces decision-support values such as predicted FWI and high-risk probability. The detection module does not estimate FWI; instead, it examines camera frames and confirms visible fire or smoke evidence. This separation keeps the numerical risk model and the visual evidence pipeline understandable, testable, and auditable.

The backend coordinates the system workflow and exposes the outputs needed by the

dashboard. Prediction runs, system state, and detection alerts are stored so that operational information is not lost after a browser refresh or backend restart. SQLite provides this persistence layer during the prototype stage by preserving prediction history, the latest system state, and durable detection alerts.

The dashboard acts as the operator-facing layer. It brings together risk status, live monitoring views, detection alerts, model/system information, and map context in a single interface. The drone-ready layer is included to show how future camera streams and supervised patrol demos can connect to the same monitoring workflow, but the current system does not claim full autonomous production drone deployment.

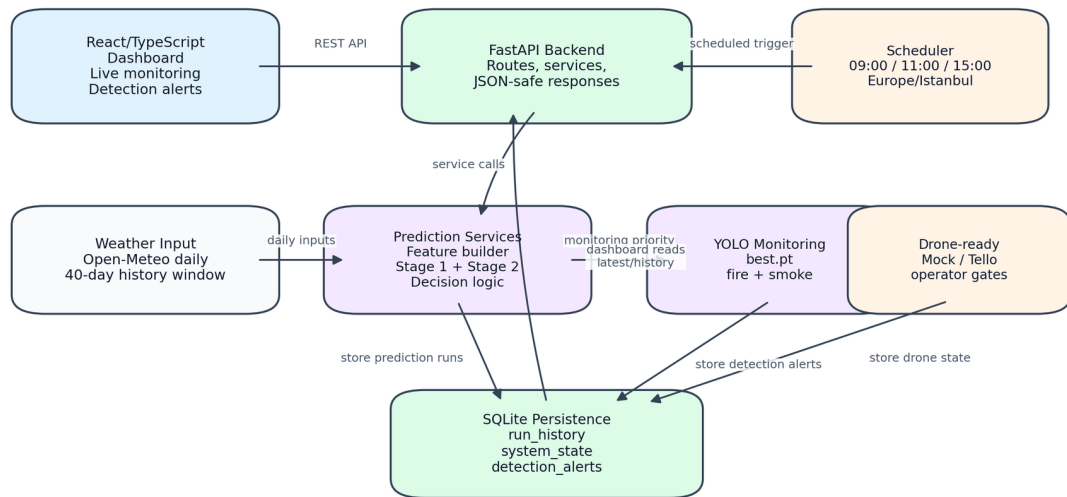


Figure 4.1: FireWatch system architecture connecting the React/TypeScript dashboard, FastAPI backend, prediction services, YOLOv8 monitoring services, scheduler, drone-ready service, and SQLite persistence.

Figure 4.1 presents the implemented architecture. The design keeps prediction services and monitoring services separate while allowing the dashboard to read both through the backend API.

4.2 Backend and API Design

The backend is implemented with FastAPI and functions as the coordination layer for the FireWatch prototype. It connects the prediction pipeline, monitoring services, detection alerting, scheduler status, drone-ready support, and dashboard data responses. This design keeps the browser client focused on presentation while the backend owns inference,

persistence, monitoring state, and API contracts.

The API is organised into high-level route groups. Risk and prediction endpoints support manual risk checks and retrieval of the latest operational prediction. Weather endpoints provide display-oriented weather information. History endpoints expose stored prediction runs for audit and review. System endpoints provide model metadata, health information, runtime configuration, and scheduler status. Monitoring endpoints manage camera feeds, detection notifications, and durable detection alerts. Drone endpoints expose operator-controlled drone-ready status, stream, demo patrol, manual command, and emergency-stop behaviours.

Manual and scheduled prediction requests use the same risk decision workflow where possible. A risk check obtains model inputs, builds the required features, runs the Stage 1 and Stage 2 models, applies the decision logic, and persists the result. Scheduled runs are triggered by the backend scheduler, while manual runs are initiated by an operator. Both run types are treated as operational prediction events and can be reviewed later through the stored history.

Camera streams and detection outputs are handled through the monitoring services rather than through the prediction route group. Local cameras, webcam/PC-camera feeds, and drone-ready video streams use the same general fire/smoke detection path. When the detector raises an event, the alerting service stores the label, confidence, source, timestamp, and optional snapshot evidence for dashboard review.

The backend structure separates major responsibilities so that model inference, camera monitoring, alert persistence, scheduler operation, drone-ready controls, and response formatting remain manageable. This separation is important for a prototype that combines several engineering areas, because it reduces coupling between the risk model and the visual monitoring layer. The design is local and prototype-oriented; it does not claim cloud deployment, production-grade security, or integration with official emergency-response infrastructure.

4.3 Database and Persistence Design

FireWatch uses SQLite as a lightweight persistence layer for the local prototype. SQLite is suitable for this stage because it is serverless, easy to inspect, and sufficient for a single-machine academic demonstration. The active runtime database is stored at `backend/outputs/karabuk_fwi.db`.

The database preserves three main categories of information. The `run_history` table stores prediction runs, including run type, timestamp, target date, predicted FWI, high-risk probability, final decision, decision reason, thresholds, raw inputs, engineered features, run metadata, and diagnostic details. The `system_state` table stores small persistent state values, such as the latest drone-ready policy state. The `detection_alerts` table stores durable fire/smoke alerts from the monitoring layer.

Persistent storage is important because FireWatch is intended to support operator review, not only live display. Prediction results should remain available after refresh or restart so operators can inspect previous runs. Detection alerts also need review status because an alert may be unread, opened, marked as read, returned to unread, or hidden from normal dashboard views. This makes persistence part of the decision-support design rather than only a technical storage detail.

Detection alerts preserve the visual evidence metadata needed by the dashboard. Stored fields include the fire/smoke label, confidence value, source camera or device, timestamp, severity/message, detection count, detailed detection payload, optional snapshot path, read/unread state, read timestamp, soft-delete state, and deletion timestamp. Snapshot images are stored as files and referenced from SQLite, while the database row keeps the operational record available for the Detection Alerts interface.

The runtime database policy treats `backend/outputs/karabuk_fwi.db` as local runtime/demo data. It is ignored by Git and should not be committed, because prediction runs, alert review state, soft-deleted alerts, and demo activity change during normal use. Shared demonstration data should therefore be prepared through seed scripts or clearly named snapshots rather than by committing the active live database.

SQLite is an appropriate choice for a B.Sc. prototype and regional pilot, but it is not presented as the final production database architecture. A multi-user or deployed monitoring system would likely require a server-based database such as PostgreSQL, stronger access control, backup policies, and operational monitoring.

Figure 4.2 shows the key runtime entities. The same SQLite database file is used for local persistence, but the monitoring layer writes only detection alert records and does not create prediction-history rows.

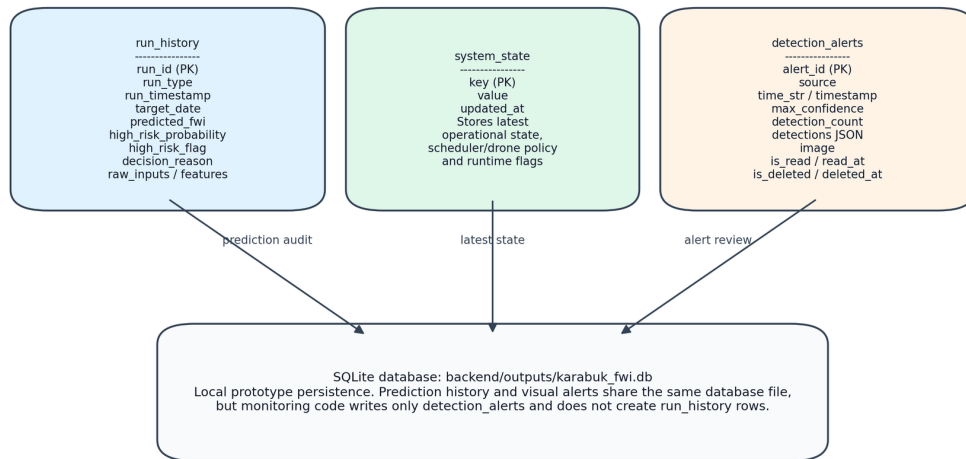


Figure 4.2: SQLite persistence design showing the main runtime tables used by FireWatch: prediction history, system state, and detection alerts.

4.4 Prediction and Detection Integration

FireWatch integrates prediction and detection as two coordinated but logically separate workflows. The prediction workflow estimates daily wildfire risk from weather-derived features using the Stage 1 and Stage 2 models. The visual detection workflow examines live camera frames using the YOLOv8 best .pt detector and records visible fire or smoke evidence. Keeping these workflows separate avoids mixing modelled risk with observed visual evidence.

The Stage 1 prediction model produces a continuous predicted FWI value, while the Stage 2 classifier supports the final high-risk decision near operationally important thresholds. This output gives the operator a risk estimate for the target date. The detection layer has a different role: it does not calculate FWI or alter the prediction result, but checks whether camera sources contain visible fire or smoke.

Both workflows meet in the backend and dashboard. Risk outputs are shown as decision-support information, while detection alerts are shown as operational evidence. This allows the operator to distinguish between a high-risk weather situation and confirmed visual signs of fire or smoke. The design therefore answers two different questions: whether conditions are dangerous, and whether there is visible evidence that requires review.

When the prediction pipeline identifies high-risk conditions, the result can support monitoring priority and drone-ready patrol recommendations. This does not mean the system autonomously launches production drone missions. Instead, the prediction result informs

the operator and prepares the monitoring layer for supervised or demo operation.

Detection alerts are persisted with practical review metadata, including the camera or device source, fire/smoke label, confidence value, timestamp, and snapshot or evidence reference when available. This gives the dashboard a durable alert history and helps operators compare risk likelihood with visual evidence from monitoring feeds.

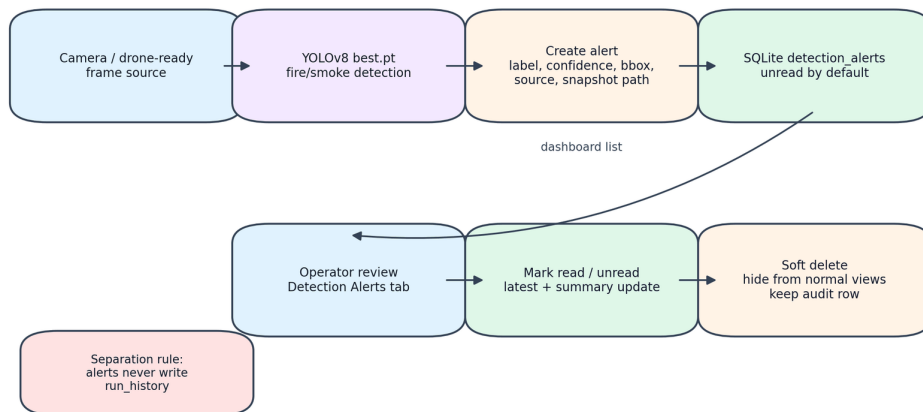


Figure 4.3: Detection alert workflow from a YOLOv8 detection event to persistent unread alerts, operator review, read/unread state, and soft deletion.

Figure 4.3 explains the alert lifecycle implemented in the dashboard. The workflow supports repeated operator review instead of treating detections as temporary notifications.

4.5 Frontend Dashboard Design

The frontend dashboard is implemented with React and TypeScript. It is the operator-facing layer of FireWatch and presents backend outputs in a structured way. The dashboard is designed to support monitoring and review rather than to duplicate backend logic. Prediction, persistence, detection, and scheduler state remain owned by the backend.

The final dashboard structure is organised around approved operational views: Overview / Live Monitoring, Detection Alerts, Risk Map, System and Model, and System Flow. These views reduce the amount of information shown at once while still giving the operator access to the main parts of the system.

The Overview / Live Monitoring view brings key information together. It is intended to show current weather and risk context, the latest operational status, live monitoring feeds,

and the most important system summary. This view acts as the starting point for daily use because it combines both risk awareness and monitoring state.

The Detection Alerts view focuses on persistent fire/smoke evidence. It supports review of alert labels, confidence values, source information, timestamps, snapshots when available, read/unread state, and soft deletion. This design makes alerts durable and reviewable rather than temporary on-screen messages.

The Risk Map provides spatial context for the Karabük monitoring area and supports planning-oriented interpretation of historical burned-area and risk context. It is used as an operator awareness view rather than as a direct replacement for the prediction model. The System and Model view summarizes model, feature, threshold, and health information needed to understand the current configuration. The System Flow view explains the end-to-end workflow visually, showing how weather input, prediction, detection, persistence, and dashboard review connect.

4.6 Scheduler and Operational Run Design

The scheduler supports repeated operational risk checks during the day. In the current FireWatch design, scheduled checks are configured for 09:00, 11:00, and 15:00 Istanbul time. Using the local operational timezone keeps scheduled decisions aligned with the daily monitoring routine for Karabük.

Each scheduled run follows the same general data-to-decision path as an operator-triggered manual run. The system obtains daily model inputs, builds the required features, runs the Stage 1 and Stage 2 prediction workflow, and applies the decision logic. This consistency helps avoid different behaviour between automatic checks and manual checks.

Both manual and scheduled runs are persisted in SQLite. The `run_history` table stores prediction results and associated metadata, allowing the dashboard to show the latest operational run and retain previous decisions for later review. This shared persistence design also makes it easier to compare manual and scheduled outputs for the same operational period. This is important because risk decisions should be auditable rather than disappearing after execution.

The scheduler design also prevents operational checks from being treated as a temporary console-only process. Scheduled checks are registered as backend jobs, exposed through system status, and connected to the same persistence path used by manual checks. This makes the behaviour visible to the dashboard and easier to verify during demonstrations.

The scheduled run design is still part of a local prototype. It supports structured decision support and review, but it does not represent official emergency deployment or certified operational wildfire command infrastructure.

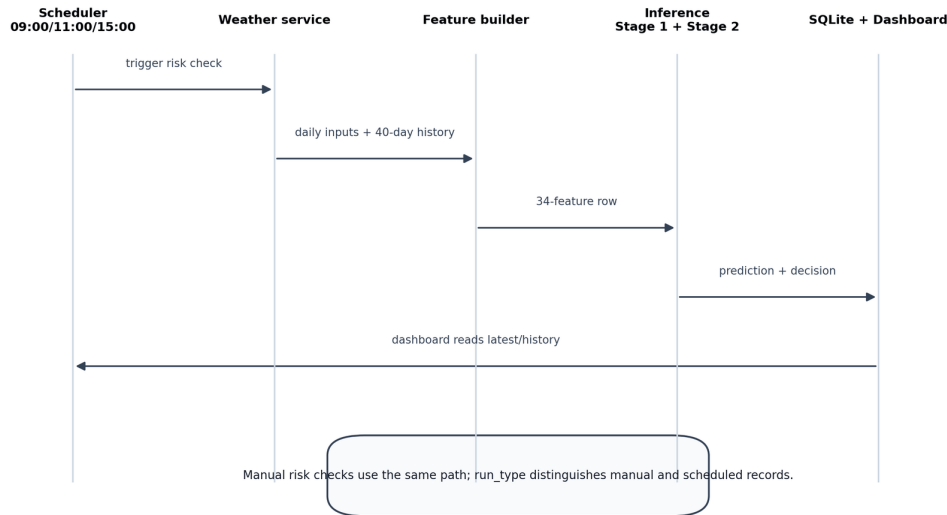


Figure 4.4: Runtime sequence for scheduled risk checks, feature generation, two-stage inference, SQLite storage, and dashboard update.

Figure 4.4 shows that scheduled runs and manual runs share the same inference and persistence path. This consistency makes operational decisions easier to audit.

4.7 Drone-Ready Implementation

FireWatch includes a drone-ready layer for supervised aerial monitoring with operator-controlled DJI Tello testing. This layer connects camera-based monitoring with a real drone video source while keeping physical drone operation under explicit operator control. It supports the project goal of being drone-ready and hardware-testable without claiming certified autonomous deployment.

The current implementation supports two execution paths: mock mode and Tello mode. Mock mode remains available for development, demonstrations without hardware, interface checks, and automated testing. Tello mode allows the system to work with a configured DJI Tello drone, including connection status, video stream handling, battery/status reporting, emergency stop support, and a short controlled demo patrol when the required safety flags are enabled.

Drone-ready support is connected to the monitoring and dashboard workflow. The system

can expose drone status, stream state, patrol recommendation state, and demo patrol results to the operator interface. The same fire/smoke detection path can process frames from the drone-ready video source, allowing the Tello stream to reuse the alerting and persistence design already used for local camera feeds.

Physical movement is protected by safety gates. Manual controls are blocked by default unless explicitly enabled. Takeoff or demo movement requires explicit configuration, sufficient battery status, an active Tello connection, and operator confirmation. Emergency stop support is available through the drone-ready API layer. This design keeps the prototype safe for academic demonstration and prevents risk prediction from directly causing uncontrolled hardware movement.

High-risk prediction can support patrol recommendation or monitoring priority, but the current project does not claim full autonomous production launch. Full waypoint navigation, return-to-home behaviour, battery-aware mission planning, regulatory compliance, and production safety validation remain outside the current scope.

4.8 Implementation Summary

This chapter described how FireWatch is implemented as a working software prototype. The system connects weather-based prediction, fire/smoke detection, SQLite persistence, dashboard presentation, scheduled risk checks, and drone-ready support into one decision-support workflow for the Karabük regional pilot.

The FastAPI backend coordinates the main workflows and exposes the data needed by the dashboard. SQLite preserves prediction history, system state, and detection alerts so that operational information remains available for review. The React/TypeScript dashboard presents risk information, alert evidence, system/model status, workflow context, and spatial awareness to the operator.

The implementation is modular: prediction, detection, persistence, scheduler operation, drone-ready support, and dashboard views are separated into clear responsibilities. This makes the prototype easier to test, explain, and extend. It also supports the methodology described in Chapter 3 and prepares the basis for the results and discussion presented in Chapter 5.

Chapter 5

Results and Discussion

5.1 Stage 1 Regression Results

The Stage 1 model was evaluated as a continuous Fire Weather Index (FWI) regression model. Its task is to estimate daily FWI from the locked set of 34 engineered input features described in Chapter 3. The deployed model is a `HistGradientBoostingRegressor`; its output, `predicted_fwi`, is the numerical risk estimate shown in the dashboard and passed to the high-risk decision layer.

Table 5.1 reports the verified Stage 1 metrics from the repository metadata. The out-of-fold validation result was $RMSE = 4.111$, $MAE = 2.918$, and $R^2 = 0.835$. The 2025 holdout result was $RMSE = 7.259$, $MAE = 5.098$, and $R^2 = 0.819$. The increase in error on the later holdout period is expected because it tests the model on a chronologically later season, but the holdout R^2 still indicates that the model captures much of the variation in the FWI target.

Table 5.1: Stage 1 regression performance for continuous FWI prediction.

Evaluation setting	RMSE	MAE	R^2	Interpretation
Out-of-fold validation	4.111	2.918	0.835	Temporal validation result across training folds; shows strong fit while respecting chronological evaluation.
2025 holdout test	7.259	5.098	0.819	Later-season holdout result; error increases, but the model still captures most variation in the FWI target.

Figure 5.1 shows that the selected regression approach follows the general trend of observed FWI values. The result should not be interpreted as exact fire-danger reproduction; instead,

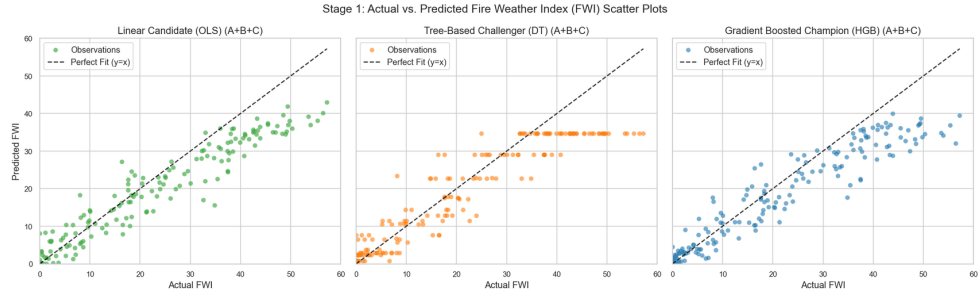


Figure 5.1: Stage 1 actual versus predicted FWI comparison for candidate regression models.

it is a decision-support estimate that preserves useful information about how close a day is to the high-risk boundary.

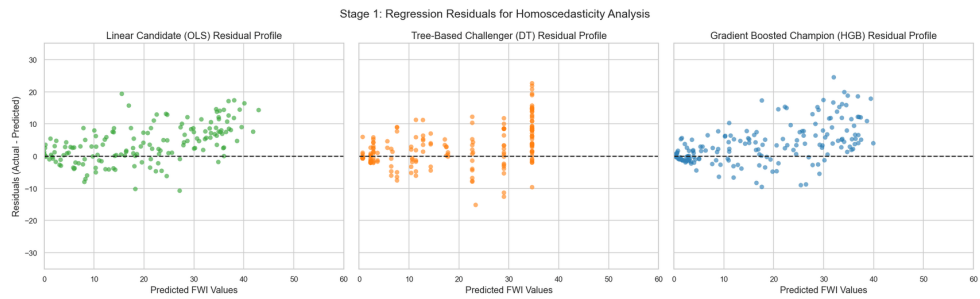


Figure 5.2: Stage 1 residual analysis showing prediction error behaviour across FWI values.

The residual plot in Figure 5.2 is important because wildfire risk decisions are sensitive to larger errors near high FWI values. The model is useful for daily risk estimation, but the residual pattern also supports the need for a second safety-oriented classification layer.

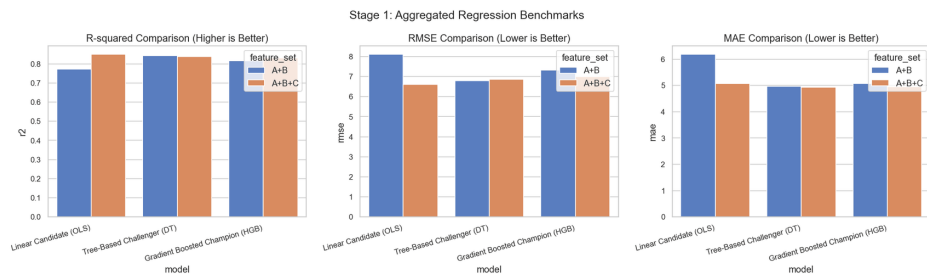


Figure 5.3: Regression benchmark comparison using R^2 , RMSE, and MAE.

Figure 5.3 summarizes the Stage 1 benchmark evidence. The combined use of RMSE, MAE, and R^2 is necessary because a single metric cannot describe both average error and explained variation.

5.2 Stage 2 High-Risk Classification Results

Stage 2 was evaluated as a high-risk classification layer. It uses a `RandomForestClassifier` with four inputs: `predicted_fwi`, `rh`, `ws`, and `fuel_drying_rate`. The class target is `target_ge_35`, where a day is high risk when $\text{FWI} \geq 35$. The final stored probability threshold is 0.10.

Table 5.2 gives the final classifier values verified from the repository metadata. At threshold 0.10, the Stage 2 classifier achieved precision = 0.849, recall = 0.900, F1-score = 0.874, and accuracy = 0.929, with 45 true positives, 8 false positives, 5 false negatives, and 126 true negatives.

Table 5.2: Final Stage 2 high-risk classification result at probability threshold 0.10.

Threshold	Precision	Recall	F1-score	Accuracy	TP	FP	FN	TN
0.10	0.849	0.900	0.874	0.929	45	8	5	126

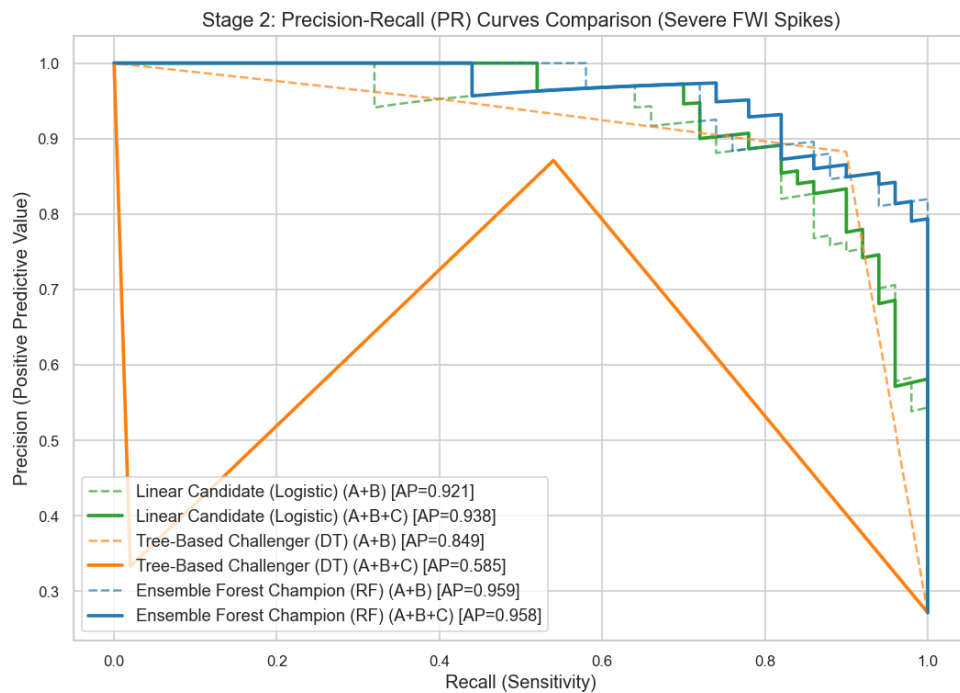


Figure 5.4: Stage 2 precision–recall comparison for imbalanced high-risk classification.

Figure 5.4 is more informative than accuracy alone because high-risk days are less frequent than normal days. In this application, recall is especially important: a false negative may reduce attention on a genuinely dangerous day. Precision still matters because too many false alarms can reduce operator trust and increase monitoring load.

The confusion matrix in Figure 5.5 uses the final report values, not the notebook ablation figure at threshold 0.25. This distinction is important because mixing ablation results with

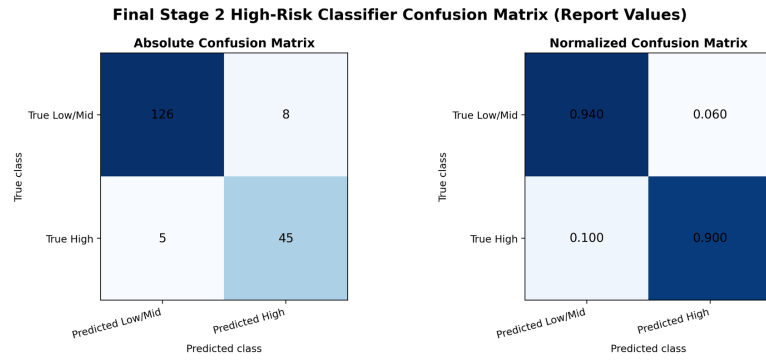


Figure 5.5: Final Stage 2 high-risk classifier confusion matrix using the report threshold of 0.10.

the final deployed threshold would misrepresent the classifier performance.

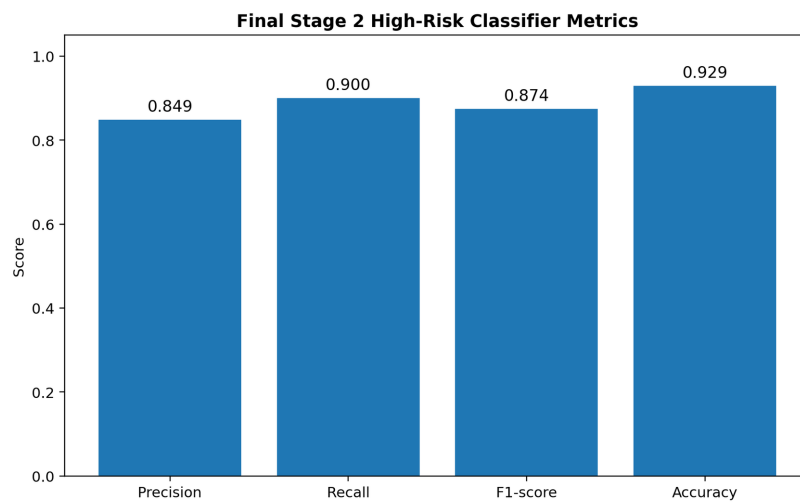


Figure 5.6: Final Stage 2 classifier metrics at threshold 0.10.

Figures 5.6 and 5.7 show that the final decision layer is not a black-box replacement for the regression model. It is a compact safety layer that uses the Stage 1 output plus humidity, wind, and fuel-drying information to reduce missed high-risk cases near the decision boundary.

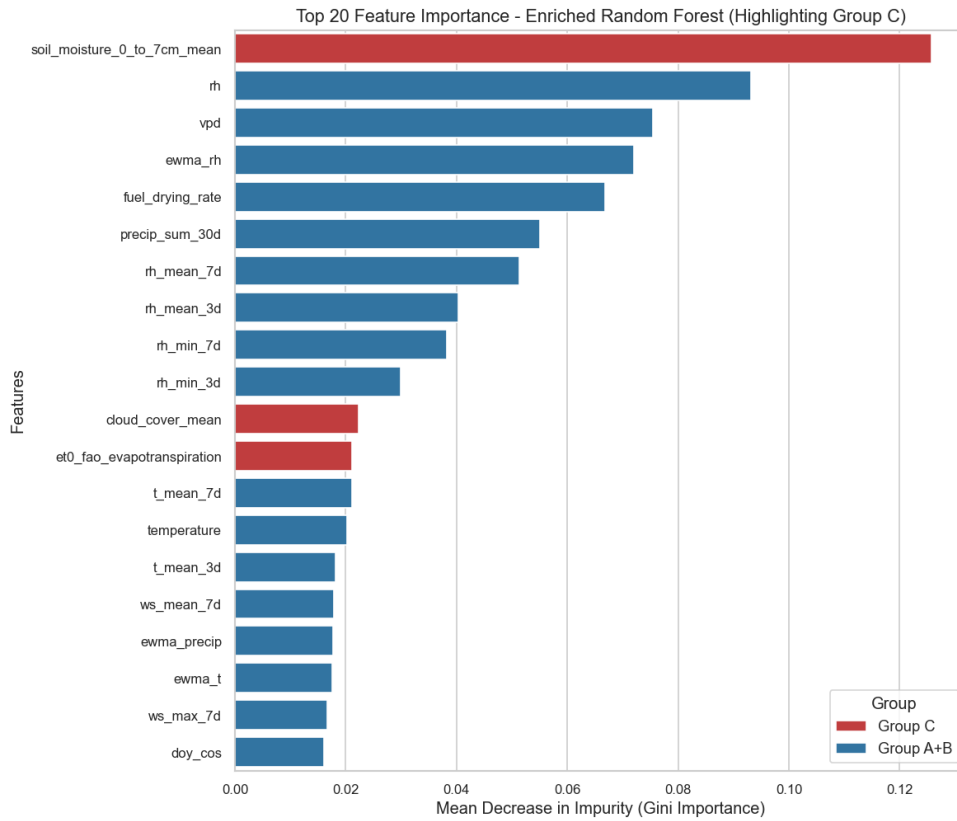


Figure 5.7: Feature importance evidence for the enriched Random Forest classifier, highlighting the contribution of the support features.

5.3 YOLOv8 Fire and Smoke Detection Results

The visual monitoring stage uses the active YOLOv8 best .pt detector for two object classes: fire and smoke. The detector provides visual evidence through bounding boxes, class labels, and confidence values. This is different from the prediction model: prediction estimates fire-weather risk, while YOLOv8 detection identifies visible evidence in camera or drone-ready frames.

Object detection should be evaluated with object-detection metrics rather than generic image-classification accuracy. Precision measures how many detections are correct, recall measures how many relevant objects are found, and $mAP@0.5$ / $mAP@0.5:0.95$ summarize localisation and classification quality across confidence and overlap thresholds. Table 5.3 therefore reports precision, recall, and mAP values from the uploaded YOLOv8 training/evaluation output.

Figure 5.8 provides the main training evidence for the available detector run. The uploaded metric summary reports best observed precision = 0.8067, best recall = 0.7401, best $mAP@0.5$ = 0.7905, and best $mAP@0.5:0.95$ = 0.4558. The final epoch values were precision = 0.8004, recall = 0.7298, $mAP@0.5$ = 0.7806, and $mAP@0.5:0.95$ = 0.4492.

Table 5.3: YOLOv8 fire/smoke detection metrics from the uploaded training and evaluation output.

Metric row	Precision	Recall	mAP@0.5	mAP@0.5:0.95	Notes/source
Best observed values	0.8067	0.7401	0.7905	0.4558	Best values reported across the provided YOLO training log summary.
Final epoch values	0.8004	0.7298	0.7806	0.4492	Final row from the provided YOLO training log summary. These values describe object-detection performance, not classification accuracy.

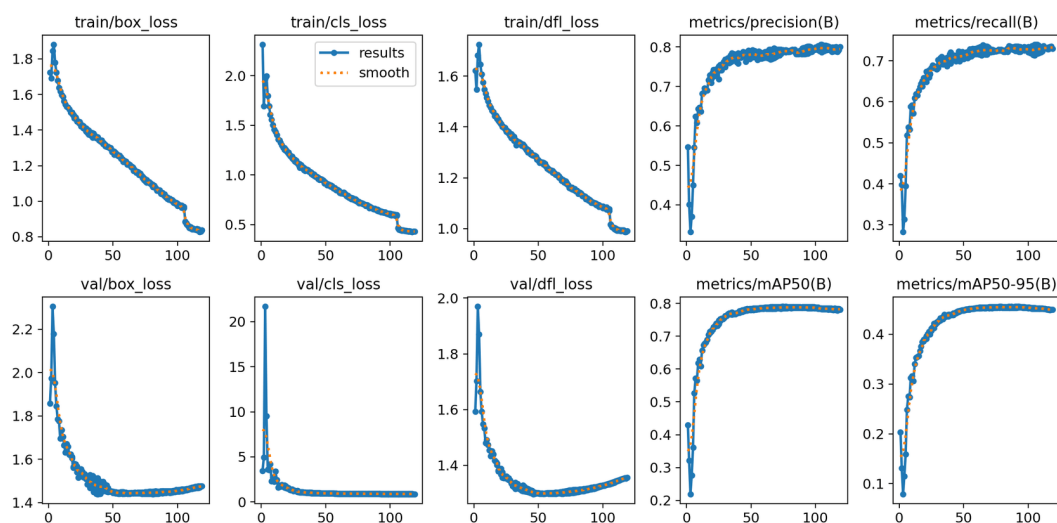


Figure 5.8: YOLOv8 training and validation curves for loss and detection metrics.

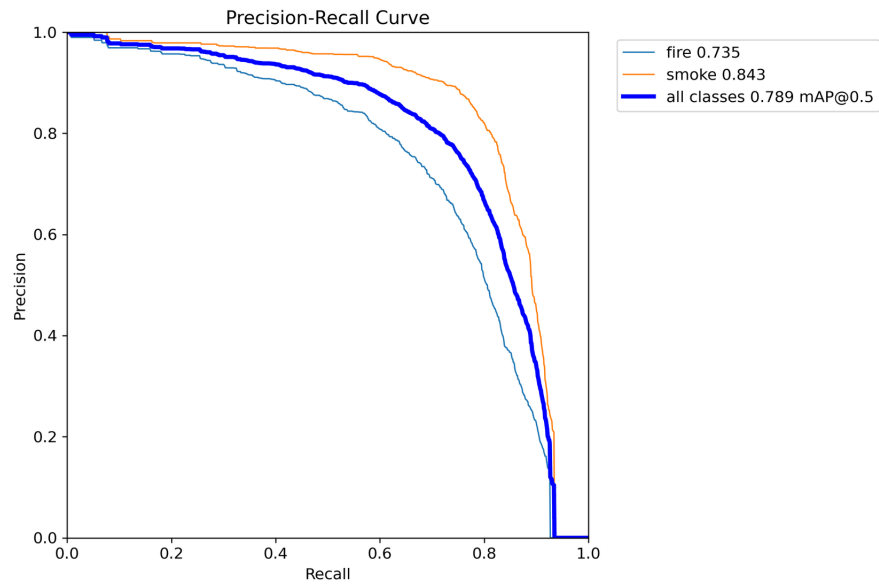


Figure 5.9: Precision–recall curve for fire and smoke detection.

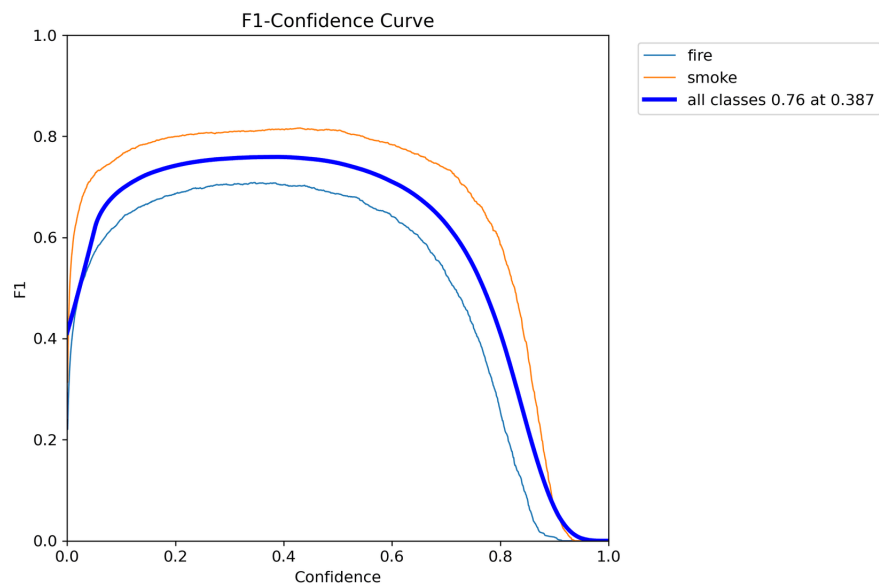


Figure 5.10: F1-confidence curve used to inspect detection threshold behaviour.

Figures 5.9 and 5.10 help explain threshold behaviour. Lower thresholds can improve recall but may increase false positives, while higher thresholds may reduce false alarms but miss weaker smoke or fire evidence.

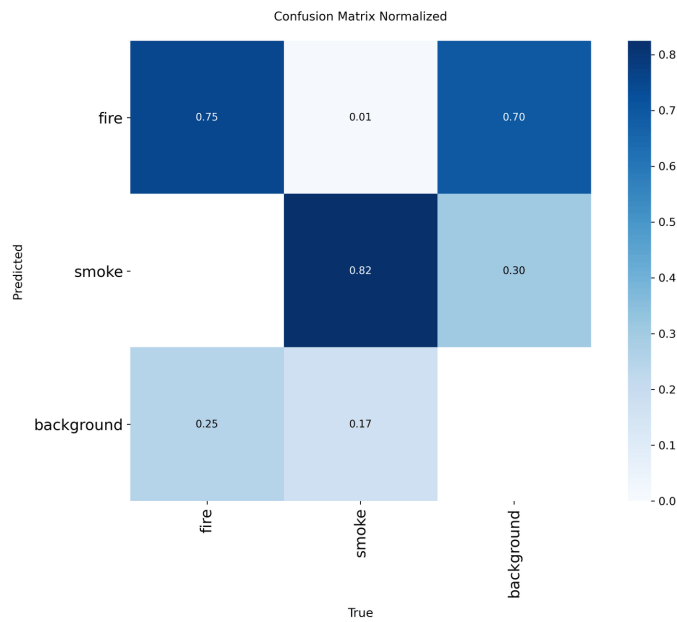


Figure 5.11: Normalized confusion matrix for fire, smoke, and background predictions.

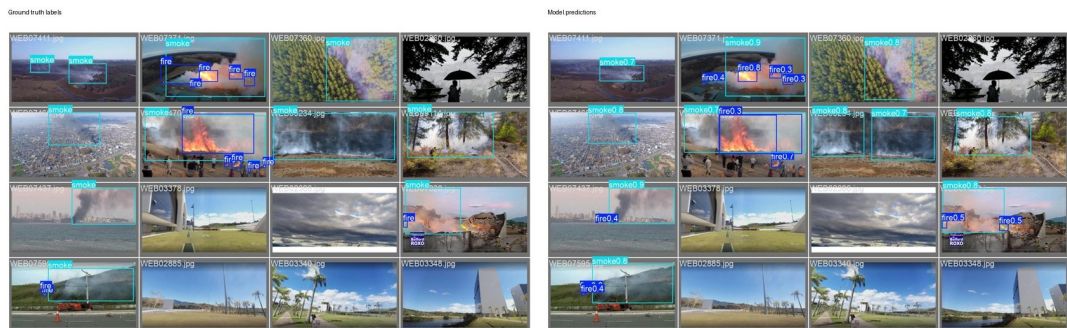


Figure 5.12: Example validation images comparing ground-truth labels with YOLOv8 predictions.

The normalized confusion matrix in Figure 5.11 and the qualitative examples in Figure 5.12 show how the detector behaves on the available evaluation material. These figures support the prototype detection workflow, but they do not prove field performance under all lighting, weather, smoke-density, and camera-motion conditions. Formal field validation remains future work.

5.4 Dashboard and System Validation Results

The dashboard demonstrates how the FireWatch workflow can be used as an operator-facing decision-support interface. It brings together risk estimates, monitoring feeds, detection alerts, system/model status, and map context instead of leaving each component as a separate tool.

The main validation result from the software layer is integration. Backend tests verify API behaviour, inference, decision logic, scheduler behaviour, monitoring endpoints, detection alert persistence, read/unread state, soft delete behaviour, and separation between detection alerts and prediction history. A local frontend lint and production build check also confirmed that the React/TypeScript dashboard compiles successfully.

Table 5.4 summarizes the system validation evidence. These checks demonstrate that the academic prototype works as a connected software system. They do not certify official emergency deployment, field camera reliability, or fully autonomous drone patrol.

5.5 Limitations

The Stage 1 regression model still has prediction error. Although the out-of-fold and holdout results show useful performance, unusual weather patterns and difficult high-FWI days can still produce larger errors. The predicted FWI value should therefore be interpreted as a decision-support estimate, not as a replacement for official fire-danger products.

The Stage 2 classifier improves high-risk decision support but does not remove false positives or false negatives. False positives can increase monitoring load, while false negatives are more serious because they may reduce attention during dangerous conditions. This is why FireWatch shows the probability and decision reason rather than hiding the result behind a single unexplained label.

The YOLOv8 detection stage is also limited. The uploaded training/evaluation curves provide useful object-detection evidence, but they are not a substitute for extended field validation in Karabük forests, under drone motion, varied lighting, glare, fog, and different smoke densities. Multi-frame confirmation, glare filtering, bounding-box area filtering, and larger local evaluation sets remain future work.

The persistence and drone-ready layers are intentionally prototype-oriented. SQLite is appropriate for a local single-machine demonstration, but a deployed multi-user system

Table 5.4: System validation evidence from the repository and local verification.

Component	Validation method	Confirmed behaviour	Limitation
Backend API and models	Automated backend test suite and smoke check	Prediction models load, API routes respond, and smoke check confirms key runtime artefacts.	Tests are local software checks, not field certification.
Scheduler and run persistence	Backend tests for scheduler configuration and scheduled runs	Scheduled risk checks use the same persistence path as manual risk checks.	Long-term unattended operation was not evaluated.
SQLite persistence	Database schema inspection and endpoint tests	run_history, system_state, and detection_alerts	SQLite is suitable for a local prototype, not multi-user production.
Detection alert workflow	Monitoring endpoint tests	Alerts support source filtering, latest lookup, read/unread state, and soft deletion without writing prediction history.	Visual detection robustness under field lighting remains future work.
Dashboard integration	Frontend lint and production build verification	The React/TypeScript dashboard builds and displays the integrated monitoring interface.	Usability testing with forestry operators was not conducted.

would require a server database, access control, backups, and operational monitoring. The drone layer supports mock and Tello-oriented workflows with operator safety gates; it does not claim certified autonomous drone patrol.

5.6 Discussion

The results answer the research question by showing that FireWatch can combine daily wildfire risk prediction, visual fire/smoke detection, persistent alert management, and dashboard-based decision support for the Karabük region. Each module contributes a different type of evidence. Stage 1 provides continuous fire-weather risk estimation, Stage 2 supports high-risk classification, YOLOv8 provides visual evidence, SQLite preserves operational history, and the dashboard makes the workflow reviewable.

The two-stage prediction design is useful because it separates risk intensity from high-risk decision support. A continuous FWI estimate helps operators understand severity, while the Stage 2 classifier focuses on reducing missed high-risk days near the decision boundary. This is more informative than a single binary model because it preserves both the estimated risk level and the operational decision.

The detection model adds a complementary role. Weather-based prediction cannot confirm that fire or smoke is already visible, and camera detection alone does not explain whether the wider weather conditions are dangerous. By keeping prediction and detection separate but visible in the same dashboard, FireWatch supports both preparedness and visual evidence review.

Overall, FireWatch is strongest as a regional academic decision-support prototype. It does not claim official emergency integration, field certification, or production autonomous drone operation. However, the results demonstrate a complete engineering workflow that connects machine learning, computer vision, persistence, scheduling, and dashboard monitoring into a coherent FireWatch prototype.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

This thesis presented FireWatch: A Drone-Ready Wildfire Risk Prediction and Fire/Smoke Detection System. The project was developed as a regional decision-support prototype for Karabük, Türkiye. Its purpose was to show how weather-based wildfire risk prediction, visual fire/smoke detection, persistent alert management, dashboard monitoring, scheduler support, and a drone-ready layer can be combined into one coherent software system.

The prediction methodology used May–October seasonal Karabük records from 2012 to 2025. Open-Meteo daily weather aggregates were used as accessible model inputs, while the processed dataset included a processed FWI target associated with official fire-danger data and used for supervised learning. The final prediction workflow used a two-stage structure: Stage 1 applied a `HistGradientBoostingRegressor` to estimate continuous FWI, and Stage 2 applied a `RandomForestClassifier` as a safety-oriented classifier for days where $\text{FWI} \geq 35$.

The visual monitoring part of the project used a custom YOLOv8 detector, with `best.pt` as the active fire/smoke detection model. This detector was kept separate from the weather-based prediction model. Its role was to provide visual evidence from camera or drone-ready monitoring sources, while the prediction model provided daily fire-weather risk awareness.

The system implementation connected these models through a FastAPI backend, SQLite persistence, a scheduler, and a React/TypeScript dashboard. SQLite made it possible to preserve prediction history and detection alerts, including review states such as read/unread and soft deletion. The dashboard provided an operator-facing interface where risk decisions, monitoring feeds, detection alerts, system/model information, workflow explanation, and

map context could be reviewed together.

The final system answers the research question by demonstrating that daily risk prediction, live fire/smoke detection, persistent alert management, and dashboard-based decision support can be integrated into one regional wildfire monitoring prototype. At the same time, the project remains realistic about its scope. FireWatch is a B.Sc. engineering prototype and regional pilot; it is not an official emergency response system and does not claim full autonomous production drone deployment.

6.2 Future Work

Future work should first improve the robustness of the visual detection module. The current prototype uses frame-based YOLOv8 detection, but practical outdoor use would benefit from multi-frame confirmation, glare and reflection filtering, bounding-box area filtering, and testing on a larger labelled fire/smoke validation set. The uploaded YOLOv8 evaluation output provides initial object-detection metrics, but a larger local benchmark and field validation campaign are still needed before operational reliability can be claimed.

The prediction workflow can also be extended. Future versions could include additional years, neighbouring regions, fuel-related variables, topography, land-cover information, and human-activity indicators if reliable data are available. The high-risk threshold and classifier probability threshold could also be calibrated using operational feedback. Uncertainty estimation would make the prediction output more useful by showing not only the predicted risk value, but also the confidence or uncertainty around that decision.

The drone-ready layer should be developed gradually and under clear safety constraints. Future work may include supervised waypoint patrol, geofencing, return-to-home behaviour, battery-aware mission planning, and safer mission state management. These improvements would also require regulatory compliance, operator procedures, hardware validation, and production-level safety testing before any real autonomous patrol could be considered.

A production deployment would require stronger infrastructure than the current local prototype. SQLite is suitable for demonstration and single-operator use, but a deployed multi-user system should use a server-based database such as PostgreSQL. Authentication, user roles, backups, monitoring, logging, and cloud or on-premise deployment procedures would also be required before the system could be used in an operational environment.

Data pipeline traceability is another important area for improvement. Future work should

formalize the raw Open-Meteo and CEMS/Copernicus download scripts, store reproducible data-preparation records, and document dataset versions. This would make it easier to rebuild the processed dataset, audit modelling decisions, and compare future model versions fairly.

Finally, field validation is necessary before FireWatch can be considered for real operational use. The system should be tested with outdoor camera scenes, different lighting conditions, different weather conditions, and real operator feedback. Such validation would help measure alert usefulness, identify false positive causes, and determine how the dashboard supports decision-making in real monitoring situations.

Bibliography

- [1] E. Çolak and F. Sunar, “Evaluation of forest fire risk in the mediterranean turkish forests: a case study of menderes region, izmir,” *International Journal of Disaster Risk Reduction*, vol. 45, p. 101479, 2020.
- [2] M. Göltaş, H. Ayberk, and O. Küçük, “Forest fire occurrence modeling in southwest turkey using maxent machine learning technique,” *iForest – Biogeosciences and Forestry*, vol. 17, no. 1, pp. 10–18, 2024.
- [3] B. Ekberzade *et al.*, “Up in flames: the human factor behind a megafire in mediterranean türkiye,” *npj Natural Hazards*, vol. 2, p. 65, 2025.
- [4] M. Bugarić, D. Krstinić, L. Šerić, and D. Stipaničev, “Current trends in wildfire detection, monitoring and surveillance,” *Fire*, vol. 8, no. 9, p. 356, 2025.
- [5] A. Mohapatra and T. Trinh, “Early wildfire detection technologies: a review,” *Sustainability*, vol. 14, no. 19, p. 12270, 2022.
- [6] D. Rjoub, A. Alsharoa, and A. Masadeh, “Unmanned-aircraft-system-assisted early wildfire detection with air quality sensors,” *Electronics*, vol. 12, no. 5, p. 1239, 2023.
- [7] S. Kondylatos *et al.*, “Wildfire danger prediction and understanding with deep learning,” *Geophysical Research Letters*, vol. 49, no. 17, p. e2022GL099368, 2022.
- [8] A. Shmuel and E. Heifetz, “Developing novel machine-learning-based fire weather indices,” *Machine Learning: Science and Technology*, vol. 4, no. 1, p. 015029, 2023.
- [9] A. T. Zaidi, “Predicting wildfires in algerian forests using machine learning models,” *Heliyon*, vol. 9, no. 7, p. e18064, 2023.
- [10] L. Li, A. Sali, N. K. Noordin, A. Ismail, and F. Hashim, “Prediction of peatlands forest fires in malaysia using machine learning,” *Forests*, vol. 14, no. 7, p. 1472, 2023.
- [11] A. Sengupta and B. J. Woodford, “Recent advances in explainable machine learning models for wildfire prediction,” *Applied Computing and Geosciences*, vol. 27, p. 100266, 2025.

- [12] N. Anastasiou, S. Kondylatos, and I. Papoutsis, “Wildfire spread forecasting with deep learning.” arXiv preprint arXiv:2505.17556, 2025.
- [13] A. Bouguettaya, H. Zarzour, A. M. Taberkit, and A. Kechida, “A review on early wildfire detection from unmanned aerial vehicles using deep learning-based computer vision algorithms,” *Signal Processing*, vol. 190, p. 108309, 2022.
- [14] M. Mukhiddinov, A. B. Abdusalomov, and J. Cho, “A wildfire smoke detection system using unmanned aerial vehicle images based on the optimized yolov5,” *Sensors*, vol. 22, no. 23, p. 9384, 2022.
- [15] M. N. A. Ramadan *et al.*, “Towards early forest fire detection and prevention using ai-powered drones and the iot,” *Internet of Things*, vol. 27, p. 101248, 2024.
- [16] N. Abramov, Y. Emelyanova, V. Fralenko, and V. Khachumov, “Intelligent methods for forest fire detection using unmanned aerial vehicles,” *Fire*, vol. 7, no. 3, p. 89, 2024.
- [17] A. Mamadmurodov *et al.*, “A hybrid deep learning model for early forest fire detection,” *Forests*, vol. 16, no. 5, p. 863, 2025.
- [18] L. He, Y. Zhou, L. Liu, Y. Zhang, and J. Ma, “Research and application of deep learning object detection methods for forest fire smoke recognition,” *Scientific Reports*, vol. 15, p. 16328, 2025.
- [19] B. J. Stocks *et al.*, “The canadian forest fire danger rating system: an overview,” *Forestry Chronicle*, vol. 65, no. 4, pp. 258–265, 1989.
- [20] J. S. Almeida, C. Huang, F. G. Nogueira, S. Bhatia, and V. H. C. de Albuquerque, “Edgefiresmoke: a novel lightweight cnn model for real-time video fire–smoke detection,” *IEEE Transactions on Industrial Informatics*, vol. 18, no. 11, pp. 7889–7898, 2022.
- [21] F. Abid and N. Izeboudjen, “Predicting forest fire in algeria using data mining techniques: case study of the decision tree algorithm,” in *Advanced Intelligent Systems for Sustainable Development (AI2SD’2019)*, Springer, 2020.
- [22] B. Dwyer *et al.*, “Roboflow (version 1.0) [software],” 2022.